

1 Cel projektu

Celem projektu było stworzenie generatora funkcji opartego na układzie LPC1768. Generator funkcji miał spełniać następujące założenia:

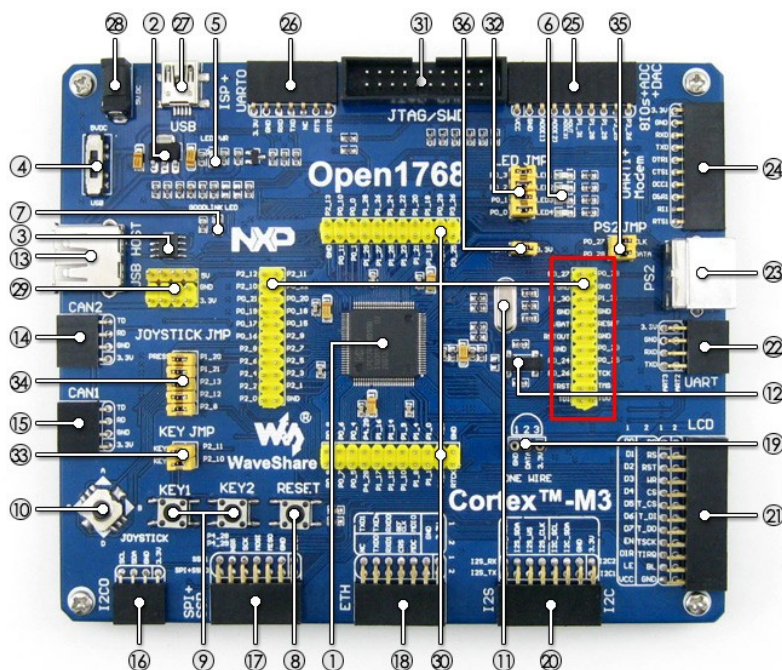
- możliwość zmiany parametrów generowanej funkcji (amplituda, częstotliwość),
- możliwość sterowania generatorem przez interfejs UART,
- możliwość wygenerowania funkcji z przebiegu narysowanego na ekranie LCD.

2 Wykorzystany sprzęt i podłączenie

Do realizacji projektu wykorzystano:

- płytkę Open1768,
- wyświetlacz LCD z ekranem dotykowym (sterownik ILI9325),
- moduł UART,
- oscyloskop do podglądu przebiegu generowanej funkcji.

Dzięki swojej modułowej budowie płytke Open1768 można łatwo rozszerzyć podłączając odpowiednie moduły do portów. **Rysunek 2.1** przedstawia płytke Open1768 wraz z oznaczeniami portów.



Rysunek 2.1: Rysunek przedstawiający płytke Open1768 z oznaczeniami portów.

Programator należy podłączyć do portu JTAG oznaczonego numerem 31. Moduł UART należy wpiąć w port UART0 oznaczony numerem 26. Moduł wyświetlacza LCD w port LCD oznaczony numerem 21. Oscyloskop należy podpiąć do pinu P0_26 oraz masy. Potrzebne piny znajdują się w złączu zaznaczonym na **Rysunku 2.1** czerwonym prostokątem. Konfigurując połączenie UART,

należy ustawić odpowiedni port COM oraz parametry transmisji, tak jak przedstawia to **Tabela 2.1**.

Tabela 2.1: Tabela zawierająca ustalone wartości parametrów transmisji UART.

Parametr	Wartość
Prędkość transmisji	115200 [Bd]
Bitów danych	8
Bitów stopu	1
Parzystość	Brak
Kontrola transmisji	Brak

Po podłączeniu modułów, oscyloskopu oraz programatora, należy wgrać opracowany program na płytkę. W tym celu należy:

1. pobrać i rozpakować archiwum z kodem projektu,
2. otworzyć projekt w środowisku μ Vision,
3. włączyć zasilanie płytki,
4. zbudować oraz wgrać program na płytkę.

3 Użytkowanie

Opracowany generator funkcji po włączeniu generuje funkcję sin z częstotliwością 10kHz o amplitudzie 3,3 V. Generator można obsługiwać przez połączenie UART, wysyłając odpowiednie komendy, lub przez wyświetlacz LCD z ekranem dotykowym.

3.1 UART

Generator można kontrolować wysyłając komendy przez interfejs UART. Każda komenda jest zatwierdzana znakiem nowej linii lub powrotu karetki. Argumenty liczbowe można przysyłać w postaci dziesiętnej, binarnej, ósemkowej lub szesnastkowej. Rozpoznawanie komend uwzględnia wielkość znaku, "help" nie jest równe "HELP". Spis obsługiwanych komend oraz ich funkcję przedstawiono w **Tabeli 3.1**.

Tabela 3.1: Spis komend i funkcji.

Typ	Komenda	Funkcja
standard	HELP	Wysyła na UART wiadomość pomocniczą, zawierającą spis wszystkich komend wraz z ich opisem
standard	START	Rozpoczyna generowanie przebiegu wcześniej ustawionej funkcji
standard	STARTBMP	Rozpoczyna generowanie wcześniej ustawionej bitmapy
standard	STOP	Zatrzymuje generowanie przebiegu
standard	<WAVE>:<FREQ>:<AMP>	Rozpoczyna generowanie funkcji WAVE o częstotliwości FREQ [Hz] z amplitudą AMP [mV]. Dozwolone wartości parametrów WAVE, FREQ, AMP przedstawiono w Tabeli 3.2
sequence	BITMAP	Rozpoczyna wczytywanie bitmapy, po wczytaniu wszystkich wierszy generuje bitmapę na ekranie oscyloskopu. Aby przerwać wczytywanie należy wysłać znak 'Q'.

Tabela 3.2: Dozwolone wartości oraz opis parametrów dla generowania funkcji przez komendę wysyłaną przez UART.

Parametr	Dozwolone wartości	Opis
WAVE	SIN, SQUARE, TRIANGLE	Kształt generowanej funkcji
FREQ	wartość całkowita z zakresu [10; 20 000]	Częstotliwość funkcji
AMP	wartość całkowita z zakresu [100; 3300]	Amplituda funkcji

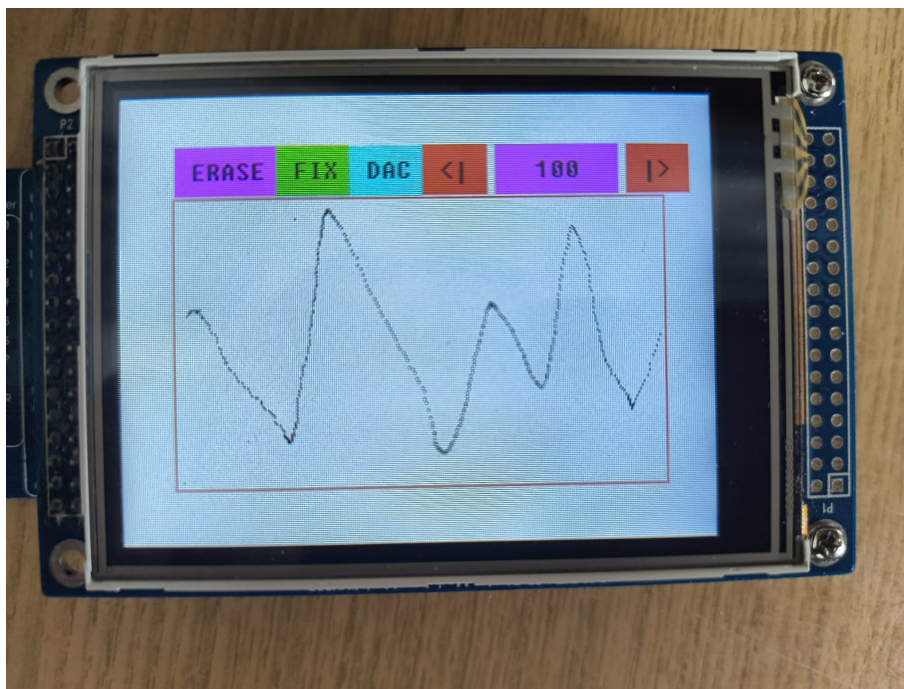
W przypadku niepoprawnej lub błędnej składniowo komendy, użytkownik zostanie poinformowany o błędzie. Wartości parametrów FREQ oraz AMP wykraczające poza zakres zostaną obcięte do dozwolonej wartości maksymalnej lub minimalnej. Komenda BITMAP wymaga specjalnych ustawień oscyloskopu. Aby poprawnie wyświetlić bitmapę należy:

1. Ustawić skalę czasową na $1,04 \left[\frac{\text{ms}}{\text{kratka}} \right]$.
2. Ustawić przesunięcie osi czasu na 5 [ms].
3. Ustawić skalę napięcia na $250 \left[\frac{\text{mV}}{\text{kratka}} \right]$.
4. Ustawić przesunięcie osi napięcia na sam dół ekranu.
5. Ustawić wyzwalacz na ≈ 2.4 [V].
6. Włączyć tryb persistence (display -> poświata), tryb zmienny 1 [s].

Aby uniknąć ryzyka epilepsji zaleca się używanie trybu single po wygenerowaniu bitmapy. Rozmiar bitmapy to 8x8. Po wywołaniu komendy BITMAP użytkownik zostanie poproszony o przesłanie ośmiu wartości liczbowych w zakresie od 0 do 255.

3.2 LCD

Korzystając z ekranu LCD można narysować oraz wygenerować własną funkcję. Do poprawnego działania ekranu dotykowego wymagana jest kalibracja. Kalibracja jest wymagana tylko raz, po wgraniu programu na nowe urządzenie. Aby uruchomić konfigurację, należy kliknąć przycisk KEY2 na płytce Open1768 a następnie zresetować płytkę klikając przycisk RESET. Ekran wyświetlacza po uruchomieniu programu przedstawiono na **Rysunku 3.1**.



Rysunek 3.1: Widok ekranu LCD po uruchomieniu programu.

Na wyświetlaczu znajdują się przyciski ERASE, FIX, DAC, przyciski odpowiadające zmianie częstotliwości generowanej funkcji oraz pole do narysowania przebiegu funkcji. Funkcję przycisków przedstawiono w **Tabeli 3.3**.

Tabela 3.3: Funkcja przycisków na ekranie LCD.

Przycisk	Funkcja
ERASE	Czyszczenie przestrzeni rysowania
FIX	"Naprawa" narysowanej funkcji. Jeśli w kolumnie znajduje się więcej niż jeden narysowany pixel, wtedy wybrany zostanie ten który jest najwyżej w kolumnie. Łączy puste przestrzenie między punktami wykorzystując interpolację liniową. Uzupełnianie przy lewej lub prawej krawędzi obszaru rysowania odbywa się za pomocą dorysowania poziomej linii na wysokości pixela który został znaleziony najbliżej tej krawędzi
DAC	Rozpoczyna generowanie narysowanej funkcji

Oprócz wyżej wymienionych przycisków, na ekranie znajdują się dwa przyciski przypominające strzałki, służące do zmiany częstotliwości narysowanej funkcji. Po kliknięciu w te przyciski wartość częstotliwości w Hercach wyświetlana między nimi będzie się zmieniać.

4 Wykorzystane peryferia

Do zrealizowania projektu wykorzystano następujące peryferia mikrokontrolera.

4.1 UART

Do komunikacji z PC wykorzystano UART0, parametry transmisji ustawiono według **Tabeli 2.1**. Do odczytywania komend przychodzących wykorzystano przerwanie RDA, ustawiono aktywację przerwania po odbiorze jednego znaku.

4.2 DAC

Do generowania funkcji wykorzystano wbudowany w mikrokontroler DAC, ustawiono podwójne buforowanie, przekazywanie przerwania do GPDMA oraz licznik działający jako timer przerwania. Modyfikując wartość w liczniku, zmienia się częstotliwość generowanej funkcji.

4.3 DMA

DMA wykorzystano w celu generowania funkcji poza "głównym wątkiem" programu. Ustawiono przesył zgodnie z **Tabelą 4.1**. Do zapętlenia transmisji wykorzystano LLI.

4.4 RTC

Wykorzystano rejestry ogólnego przeznaczenia zegara rzeczywistego do przechowywania danych związanych z kalibracją ekranu dotykowego. Rejestry te nie mają wartości domyślnej, przechowują dane nawet po resetie mikrokontrolera.

Tabela 4.1: Konfiguracja rejestrów DMA.

Parametr	Rejestr	Wartość
Adres źródła	DMACCSrcAddr	bufor przechowujący LUT funkcji
Adres docelowy	DMACCDestAddr	rejestr DACR w DAC
Rozmiar transferu	DMACCControl	ilość próbek
Rozmiar źródła	DMACCControl	16 bitów
Rozmiar docelowy	DMACCControl	16 bitów
Inkrementacja źródła	DMACCControl	włączona
Rodzaj transferu	DMACCConfig	pamięć do peryferium

4.5 SysTick

Użyto licznika SysTick do odmierzania czasu. Ustawiono SysTick, tak aby wywoływał przerwanie co jedną milisekundę.

4.6 Przerwania

Oprócz przerwania związanych z peryferiami, wykorzystano również przerwanie EINT0 obsługujące wciśnięcie przycisku KEY2, odpowiedzialnego za rozpoczęcie kalibracji ekranu dotykowego. Jako że procedurę kalibracji wywołuje się bezpośrednio w przerwaniu, ustawiono najniższy możliwy priorytet tego przerwania w NVIC.

5 Wykorzystane algorytmy

Wszystkie wykorzystywane peryferia, z wyjątkiem SysTick, są konfigurowane przez odpowiednie rejestry mikrokontrolera. SysTick skonfigurowano wykorzystując CMSIS. Do obsługi wyświetlacza i ekranu dotykowego wykorzystano bibliotekę używaną na zajęciach laboratoryjnych. Działanie programu podzielono na główną pętlę oraz przerwania. Główna pętla programu odpowiada za interakcję z wyświetlaczem, przerwania za wywołanie kalibracji i obsługę UART. Ponadto dzięki wykorzystaniu DMA generowanie funkcji odbywa się "w tle". Poniżej omówiono kolejno szczegółowe działanie kolejno generowania funkcji, obsługi UART, kalibrację ekranu dotykowego oraz obsługę wyświetlacza.

5.1 Generator funkcji

Funkcje są generowane na podstawie tablic (LUT) zawierających wcześniej przygotowane wartości próbek. Każda funkcja składa się z 50 próbek, wartości funkcji są przeskalowane do zakresu 10 bitowego DAC, są więc w zakresie [0; 1023]. Do generowania LUT przygotowano skrypt Python generujący plik, który zawiera poprawnie składniowo tablice w C. Zdecydowano się na 50 próbek ze względu na powiązanie ilości próbek a zakres częstotliwości generowanych funkcji. DAC pracuje z maksymalną częstotliwością 1MHz. Częstotliwość pracy DAC f_{DAC} można modyfikować poprzez zmianę wartości jego licznika używanego do generowania przerwania oraz aktualizowania wartości w wewnętrznym rejestrze DAC z bufora. Częstotliwość pracy DAC wyraża się wzorem:

$$f_{DAC} = \frac{f_{pclk}}{V},$$

gdzie f_{pclk} to zegar peryferium podłączony do DAC, równy 25MHz a V to wartość rejestru licznika DAC. Jeśli maksymalna częstotliwość pracy DAC wynosi 1MHz, to minimalna wartość w liczniku V_{min} wynosi 25. Licznik DAC jest 16 bitowy, więc maksymalna wartość V_{max} wynosi $2^{16} - 1 = 65535$. Częstotliwość generowanej funkcji f_{fun} wyraża się wzorem:

$$f_{fun} = \frac{f_{DAC}}{N} = \frac{f_{pclk}}{N \cdot V} = \frac{25 \cdot 10^6}{N \cdot V},$$

gdzie N to ilość próbek w funkcji. Korzystając z powyższego wzoru można obliczyć graniczne dopuszczalne częstotliwości generowanej funkcji dla liczby próbek N podstawiając za V , wartości V_{min} i V_{max} . W Tabeli 5.1 przedstawiono zakresy dopuszczalnych częstotliwości generowanej funkcji względem liczby próbek.

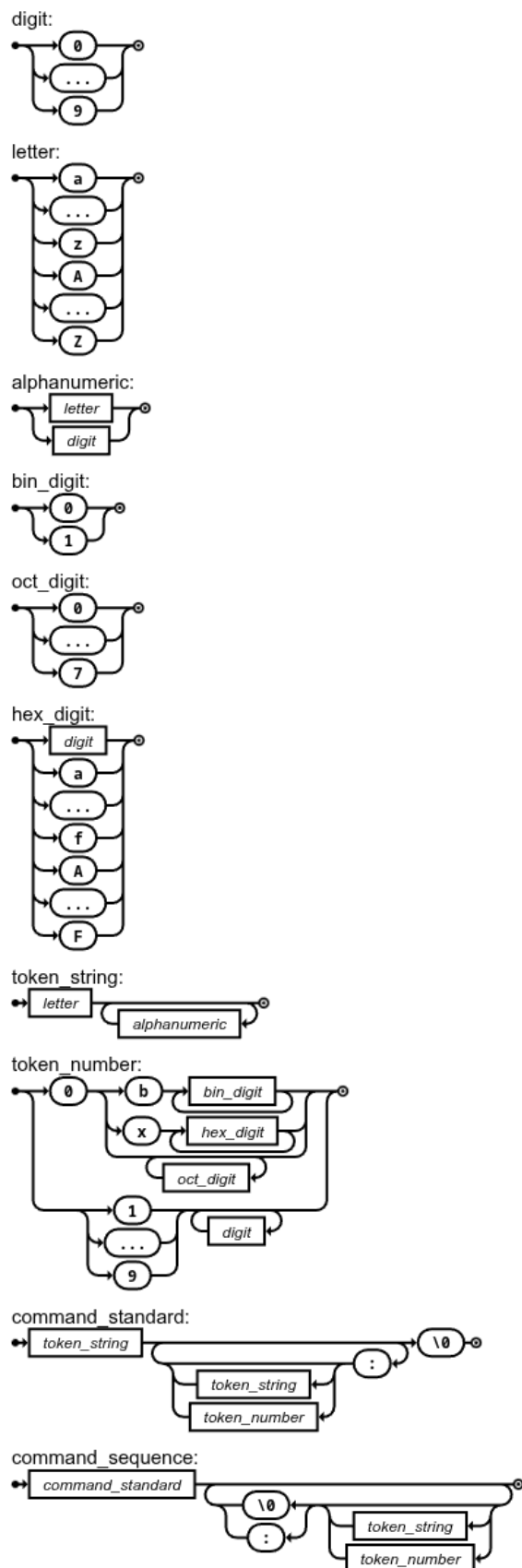
Tabela 5.1: Obliczone wartości minimalnej f_{min} oraz maksymalnej f_{max} możliwej częstotliwości generowanej funkcji dla N próbek funkcji.

N	f_{min} [Hz]	f_{max} [kHz]
10	39	100
20	20	50
50	8	20
100	4	10
150	3	6,6
200	2	5

Uznano, że 50 próbek zachowa dobrą równowagę między zakresem częstotliwości funkcji oraz rozdzielczością. Amplituda funkcji jest ustawiana przez przeskalowanie wartości podczas przepisywania wartości z LUT do bufora, z którego korzysta DMA przy przesyle. DMA korzysta wyłącznie z tego bufora, nie przesyła danych bezpośrednio z LUT. Generowanie funkcji narysowanej przez użytkownika jest realizowane przez ten sam mechanizm. Częstotliwość zależy od wartości wpisanej w licznik DAC, a wartości są przepisywane z wewnętrznej tablicy zdefiniowanej w kodzie obsługi wyświetlacza do bufora z którego korzysta DMA. Przy generowaniu funkcji z wyświetlacza ustalona jest zawsze maksymalna amplituda równa 3,3V.

5.2 UART

UART wykorzystano jako interfejs komunikacyjny z PC. W trakcie działania programu wysyłane są informacje wykorzystywane przy debugowaniu. Interfejs UART służy również do odbierania komend wysyłanych przez użytkownika. Wykorzystywano przerwanie RDA o progu jednego znaku do zbierania kolejnych znaków w tablicy. Po napotkaniu znaku nowej linii '\n' lub powrotu karetki '\r', komenda jest parsowana. Autorski parser przyjmuje komendy o składni przedstawionej na **Rysunku 5.1**.



Rysunek 5.1: Diagram kolejkowy poprawnej składni komendy UART. Znaki '\0' są wstawiane zamiast znaku końca linii podczas odbierania komendy.

Parsowanie odbywa się poprzez czytanie kolejnych tokenów, oddzielonych znakiem ':'. Pierwszy token jest nazwą komendy. Kolejne tokeny (maksymalnie 2) są traktowane jako argumenty, w przypadku gdy jest to reprezentacja tekstowa liczby, token jest konwertowany na liczbę całko-

witą. Po sparsowaniu całej komendy, odpowiednia funkcja jest wywoływana wraz z argumentami. Każda zdefiniowana funkcja sprawdza poprawność argumentów. Mapowanie komend do odpowiednich funkcji odbywa się przez porównywanie elementów tablicy przechowującej struktury komend (nazwa, funkcja) do tokenu określającego nazwę komendy. Wykorzystano przekazywanie argumentów do funkcji stylem funkcji `main` z C (`argc`, `argv[]`). Dzięki temu możliwa jest implementacja nowych komend przyjmujących więcej argumentów, nie modyfikując kodu parsera. Do obsługi komendy BITMAP, ładującej bitmapę do pamięci mikrokontrolera stworzono osobny typ komendy - `command_sequence`. W przeciwieństwie do zwykłej komendy ten typ umożliwia przekazywanie argumentów do funkcji w osobnych wywołaniach parsera. Po wywołaniu komendy tego typu parser traktuje wszystkie następne tokeny jako argumenty, niejako przekazuje kontrolę nad wykonaniem do funkcji obsługującej daną komendę. Dana funkcja po poprawnym zakończeniu musi zwrócić odpowiednią wartość aby zresetować stan parsera, zakończyć komendę sekwencyjną.

5.3 Kalibracja ekranu dotykowego

Do kalibracji ekranu wykorzystano transformację afiniczną umożliwiającą poprawne mapowanie punktów między układami współrzędnych wyświetlacza i ekranu dotykowego przez translację, rotację i skalowanie. Do obliczenia współczynników tej transformacji, wymagane są trzy pary punktów. Procedura kalibracyjna rysuje kolejno trzy krzyżyki w rogach ekranu oraz zbiera dane po wciśnięciu przez użytkownika danego krzyżyka. Do obliczenia współczynników transformacji, należy rozwiązać układ równań:

$$\begin{cases} l_x = At_x + Bt_y + C \\ l_y = Dt_x + Et_y + F \end{cases} \quad (5.1)$$

Wykorzystując zebrane punkty równanie dla współrzędnej x zapisano w postaci macierzowej:

$$\begin{bmatrix} t_{x1} & t_{y1} & 1 \\ t_{x2} & t_{y2} & 1 \\ t_{x3} & t_{y3} & 1 \end{bmatrix} \begin{bmatrix} A \\ B \\ C \end{bmatrix} = \begin{bmatrix} l_{x1} \\ l_{x2} \\ l_{x3} \end{bmatrix}.$$

Powyższy układ równań rozwiązano metodą Cramera, licząc wyznaczniki odpowiednich macierzy. Po podstawieniu przykładowych danych okazało się, że wyznaczniki potrzebne do rozwiązania układu są wielkości rzędu 10^7 . Z uwagi na brak jednostki FPU w mikrokontrolerze, obliczenia muszą być wykonywane wyłącznie na liczbach całkowitych. Aby uwzględnić wartości dziesiętne obliczanych współczynników $A, B, \dots$ podczas ich obliczania należało pomnożyć je przez wartość 10^p , gdzie p to precyzja obliczeń. Użyto $p = 3$, co zapewnia dokładność na poziomie jednej tysięcznej. Przy mnożeniu wyznacznika wielkości rzędu 10^7 przez 10^3 otrzymywano wielkości rzędu 10^{10} , co wykracza poza zakres 32 bitowej wartości całkowitej. Rozwiązano ten problem przez początkowe podzielenie wartości współrzędnych przez 10. Można to było zrobić z uwagi, że wartości współrzędnych ekranu dotykowego są rzędu $10^3, 10^4$ i są zaokrąglane do 10, a współrzędne wyświetlacza dobrano tak, aby były wielokrotnością 10. Docelowy układ równań wyglądał następująco:

$$\begin{bmatrix} 0.1t_{x1} & 0.1t_{y1} & 1 \\ 0.1t_{x2} & 0.1t_{y2} & 1 \\ 0.1t_{x3} & 0.1t_{y3} & 1 \end{bmatrix} \begin{bmatrix} A \\ B \\ C \end{bmatrix} = \begin{bmatrix} 0.1l_{x1} \\ 0.1l_{x2} \\ 0.1l_{x3} \end{bmatrix}.$$

Obliczane wyznaczniki macierzy zmniejszyły się do rzędu 10^4 , a więc o trzy rzędy wielkości. Podzielenie współrzędnych miało wpływ jedynie na współczynnik C dla x oraz F dla y (o 10 razy mniejsze), co zrekompensowano podczas ich obliczania. Obliczone współczynniki wykorzystywano podczas mapowania układu współrzędnych ekranu dotykowego do wyświetlacza. Podstawiano odpowiednie wartości do równania (5.1), pamiętając że obliczone współczynniki są 10^p razy większe. Po kalibracji współczynniki $A, B, \dots, F$ zapisano w rejestrach ogólnego przeznaczenia RTC, przez co kalibracja była potrzebna wyłącznie jeden raz. Przy kolejnych uruchomieniach mikrokontrolera wartości są pobierane z rejestrów.

5.4 Wyświetlacz

5.4.1 Wypisywanie tekstu wertykalnie

Wypisywanie tekstu o orientacji wertykalnej zrealizowano poprzez modyfikację rejestru wyświetlacza – `Entry Mode (R03h)` układu ILI9325. Rejestr ten pozwala tylko i wyłącznie na zapis, dlatego w kodzie zapisane są zarówno ustawienia domyślne jak i wertykalne. Funkcja `draw_text_vertical` zmienia domyślne parametry inkrementacji wskaźnika pamięci `GRAM`. Ustawiono oś pionową jako

główną oś adresowania oraz aktywowano jednoczesną inkrementację zarówno w osi poziomej, jak i pionowej.

Dla każdego pojedynczego znaku wywoływana jest funkcja `draw_char_vertical`, która za pomocą odpowiednich rejestrów definiuje okno robocze odpowiadające wymiarom znaku (8×16 pikseli). Następnie znak jest rysowany linia po linii. Wykorzystywana jest do tego mapa bitowa znaku z tablicy ASCII – kolejne piksele są zapalane lub gaszone poprzez maskowanie bitowe odczytywanych bajtów. Po wypisaniu całego zadanego łańcucha znaków przywracane są domyślne ustawienia rejestru konfiguracyjnego wyświetlacza.

5.4.2 Algorytm naprawy funkcji narysowanej na ekranie

Algorytm naprawy przebiegu (uruchamiany dedykowanym przyciskiem) ma na celu wyeliminowanie nieciągłości w narysowanej przez użytkownika linii. Działanie rozpoczyna się od wywołania funkcji `read_graph`, która skanuje zdefiniowany obszar do rysowania na ekranie. Skanowanie odbywa się wzdłuż osi pionowej ekranu (tej w której ekran jest dłuższy). Dla każdego kroku wyszukiwane jest maksymalne wychylenie czarnego piksela, co gwarantuje przypisanie dokładnie jednej wartości do każdego argumentu (eliminacja potencjalnych błędów przy rysowaniu grubych linii), co jest ważne żeby można było wogóle mówić, że to co zostało narysowane jest funkcją.

W przypadku wykrycia przerw w odczytanym przebiegu (oznaczonych jako wartość -1), stosowana jest aproksymacja w postaci interpolacji liniowej. Algorytm odnajduje najbliższe znane wartości z lewej oraz z prawej strony braku i na podstawie równania prostej oblicza punkty pośrednie. W przypadkach, gdy brakuje wartości na krawędziach obszaru rysowania, kopiowana jest najbliższa odczytana wartość. Po zakończeniu obliczeń, obszar rysowania jest czyszczony, a zrekonstruowana, ciągła funkcja jest ponownie rysowana na ekranie piksel po pikselu z wykorzystaniem przekalkulowanej tablicy.

5.4.3 Panel dotykowy a wyświetlacz

Obsługa dotyku zrealizowana jest na zasadzie aktywnego oczekiwania na zmianę stanu odpowiedniego pinu mikrokontrolera. Po wykryciu nacisku, pobierane są uśrednione wartości (funkcja `TP_get_mean_XY`), które następnie poddawane są transformacji za pomocą funkcji `TP_to_LCD`. Pozwala to na dokładne zmapowanie obszaru dotyku na fizyczne współrzędne pikseli wyświetlacza.

5.4.4 Rysowanie funkcji przez użytkownika

Rysowanie odbywa się wewnątrz głównej pętli programu. Po pomyślnym zarejestrowaniu nacisku na panel dotykowy i przeliczeniu współrzędnych, program weryfikuje, czy punkt ten znajduje się wewnątrz zdefiniowanych granic obszaru rysowania. W przypadku spełnienia tego warunku, wywoływana jest funkcja `draw_pixel`. Funkcja ta zapisuje bezpośrednio wybraną wartość koloru do pamięci `GRAM` pod adresy wskazujące na odczytane współrzędne x i y .

Ze względu na dyskretny charakter odczytów z panelu dotykowego i ograniczoną częstotliwość próbkowania, szybkie ruchy rysika skutkują powstawaniem izolowanych punktów (przerw w linii). Zjawisko to naturalnie wymusiło implementację wspomnianego wyżej algorytmu interpolacji naprawiającej ciągłość wykresu.